Lesson 8

Code examples:
https://github.com/SEIDOAB-ADOP/ADOPM3_08.git

Asynchronous programming

In asynchronous operations the work is typically done after the method has returned to the caller. Asynchronous methods typically return quickly to the caller and are non-blocking.

So far I have used *Thread.Start*() and *Task.Run()* as typical asynchronous methods.

Asynchronous programming is built into C# using keyword async and await by which the compiler hides all the thread synchronization details. You simply write and call your asynchronous methods in a resemblance to normal synchronous programming.

Asynchronous design recipe

The design recipe using async/await is:

1. Define your asynchronous method to returns *Task<TResult>* type if it returns a value. Otherwise, it should return *Task* or *void*. In either case your method uses the ***async*** keyword.
2. Name your method XXXAsync to clearly highlight it is asynchronous
3. For I/O bound code, such as streams, your method should *await* the time-consuming IO operation to complete using *await* keyword. If your method returns a value, the caller uses *Task.Result* property to access it.
4. For CPU bound code, such as time-consuming calculations, your method should start the heavy calculation in a separate task using *Task.Run()* and *await* the completion using *await* keyword.

```
internal class IOBoundAsync
{
    1 reference
    public async Task<int> GetDotNetCountAsync()
    {
        var html = await new HttpClient().GetStringAsync("https://dotnetfoundation.org");
        return Regex.Matches(html, @"\.NET").Count;
    }
}
0 references
internal class CPUBoundAsync
{
    1 reference
    public async Task DisplayPrimeCountsAsync()
    {
        for (int i = 0; i < 10; i++)
            Console.WriteLine(await GetPrimesCountAsync(i * 1000000 + 2, 1000000) +
                " primes between " + (i * 1000000) + " and " + ((i + 1) * 1000000 - 1));

        Console.WriteLine("Done!");
    }

    1 reference
    Task<int> GetPrimesCountAsync(int start, int count)
    {
        return Task.Run(() =>
           ParallelEnumerable.Range(start, count).Count(n =>
             Enumerable.Range(2, (int)Math.Sqrt(n) - 1).All(i => n % i > 0)));
    }
}
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Invoking GetDotNetCountAsync");
        int count = new IOBoundAsync().GetDotNetCountAsync().Result;
        Console.WriteLine($"Number of times .Net keyword displayed is {count}");

        Console.WriteLine("Invoking DisplayPrimeCountsAsync");
        var t1 = new CPUBoundAsync().DisplayPrimeCountsAsync();
        t1.Wait();
    }
}
```

https://github.com/SEIDOAB-ADOP/ADOPM3_08/blob/main/ADOPM3_08_01/Program.cs

The async modifier instructs the compiler to ensure there must be an await keyword in the method.

The await operator suspends evaluation of the enclosing async method until the asynchronous operation completes. When the asynchronous operation completes, the await operator returns the result of the operation, if any.

When the await operator is applied to the operand that represents an already completed operation, it returns the result of the operation immediately without suspension of the enclosing method.

When you use the await operator in an async method, it does not block the calling Task. For example, if a UI Task starts an async operation with await, the UI remains responsive. While the async method is suspended, control returns to the UI Task, which can continue processing other events and rendering graphics until the awaited task completes.

```
0 references
internal class myTasks
{
    2 references
    private async Task<string> TaskOne()
    {
        await Task.Delay(1000);
        return "task one";
    }
    2 references
    private async Task<string> TaskTwo()
    {
        await Task.Delay(2000);
        return "task two";
    }
    1 reference
    public async Task<long> Run1()
    {
        var watch = new Stopwatch();
        watch.Start();

        await TaskOne();
        await TaskTwo();

        watch.Stop();
        return watch.ElapsedMilliseconds;
    }
    1 reference
    public async Task<long> Run2()
    {
        var watch = new Stopwatch();
        watch.Start();

        var t1 = TaskOne();
        var t2 = TaskTwo();

        await t1;
        await t2;

        watch.Stop();
        return watch.ElapsedMilliseconds;
    }
}
0 references
static void Main(string[] args)
{
    var mt = new myTasks();
    long exe1 = mt.Run1().Result;
    Console.WriteLine($"Run1: {exe1}ms");

    long exe2 = mt.Run2().Result;
    Console.WriteLine($"Run2: {exe2}ms");
}
```

https://github.com/SEIDOAB-ADOP/ADOPM3_08/blob/main/ADOPM3_08_02/Program.cs

Task Cancellation and Progress reporting

To allow cancellation of an asynchronous operation use *CancellationTokenSource*(). It is supported by most asynchronous methods in the CLR and propagates nicely.

To implement progress reporting implement the *IProgress<T> interface.*

https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtokensource

```
using System.Threading.Tasks;

namespace ADOPM3_08_04a
{
    internal class CPUBoundAsync
    {
        public Task DisplayPrimeCountsAsync(IProgress<string> onProgressReporting, CancellationToken cancellationToken)
        {
            //Notice I can use async in Lambda Expression
            return Task.Run(async () =>
            {
                for (int i = 0; i < 100 && !cancellationToken.IsCancellationRequested; i++)
                {
                    int nrprimes = await GetPrimesCountAsync(i * 1000000 + 2, 1000000);
                    onProgressReporting.Report($"{nrprimes} primes between " + (i * 1000000) + " and " + ((i + 1) * 1000000 - 1));
                }
           });
        }
        Task<int> GetPrimesCountAsync(int start, int count)
        {
            return Task.Run(() =>
               ParallelEnumerable.Range(start, count).Count(n =>
                 Enumerable.Range(2, (int)Math.Sqrt(n) - 1).All(i => n % i > 0)));
        }
    }
    class Program
    {
        static async Task Main(string[] args)
        {
            Console.WriteLine("Invoking DisplayPrimeCountsAsync");

            //Define a cancellation token to allow Task to cancel
            var cancellationSource = new CancellationTokenSource();
            var cancellationToken = cancellationSource.Token;

            //Define my progressReporter as an instance of Progress which implements IProgress
            var progressReporter = new Progress<string>(value => Console.WriteLine(value));

            //Create and run the task, but passing the progressReporter as an argument
            var t1 = new CPUBoundAsync().DisplayPrimeCountsAsync(progressReporter, cancellationToken);
```

https://github.com/SEIDOAB-ADOP/ADOPM3_08/blob/main/ADOPM3_08_04a/Program.cs

Task synchronization

Task combinators *WhenAny*, *WhenAll* allows easy blocking of current thread until *Task* completion.

```
10          static async Task Main(string[] args)
11          {
12              var random = new Random();
13              IEnumerable<Task<int>> tasks = Enumerable.Range(1, 5).Select(n => Task.Run(async () =>
14              {
15                  int mysleep = random.Next(1000, 5000);
16                  Console.WriteLine($"I'm task {n} and I am sleeping {mysleep}ms");
17                  await Task.Delay(mysleep);
18                  return n;
19              }));
20
21
22              Task<Task<int>> whenAnyTask = Task.WhenAny(tasks);
23              Task<int> completedTask = await whenAnyTask;
24              Console.WriteLine("The winner is: task " + completedTask.Result);
25
26              await Task.WhenAll(tasks);
27              Console.WriteLine("All tasks finished!");
28          }
```

https://github.com/SEIDOAB-ADOP/ADOPM3_08/blob/main/ADOPM3_08_05/Program.cs

.NET provides a range of types that you can use to synchronize access to a shared resource or coordinate Task and Thread interaction.

https://learn.microsoft.com/en-us/dotnet/standard/threading/overview-of-synchronization-primitives

Async methods in .NET

.NET Core exposes hundreds Task based asynchronous methods that you can use the await operator on.

They all:

- Return a running *Task* or *Task<TResult>*
- Has “Async” suffix to the name (except in some special cases)
- Overloaded to accept *CancellationTokenSource* and *IProgress<T>* if it supports cancellation and/or progress reporting